LABORATORIO # 2

Descripción general

Este laboratorio integra **inserción** y **eliminación** en **listas simplemente enlazadas**, usando punteros y manejo de memoria dinámica en **C/C++**. Está diseñado para reforzar el razonamiento sobre casos de borde, recorridos, y la correcta reconexión de nodos.

Objetivos de aprendizaje

- Comprender la estructura de una lista simplemente enlazada (INFO y LIGA).
- Implementar inserciones (inicio y posición relativa).
- Implementar eliminaciones: inicio, último, por valor X, antes de X y después de X.
- Analizar casos de borde y complejidad temporal.
- Verificar resultados mediante impresión de la lista.

Conocimientos previos

- Punteros y memoria dinámica (malloc/free o new/delete).
- Estructuras (struct).
- Recorridos secuenciales.

Puntos a Desarrollar

- **Inserción en listas simplemente ligadas**
  - **Inserción al inicio**: crear nodo, apuntar su LIGA a la cabeza, actualizar cabeza.
  - **Inserción después de X**: buscar X; si existe, enlazar el nuevo nodo entre X y su siguiente.
- **Eliminación en listas simplemente ligadas**

- **Eliminar inicio**: mover cabeza al segundo nodo y liberar el primero (O(1)).
- **Eliminar último**: recorrer hasta el penúltimo, poner su LIGA en NULL y liberar el último (O(n)).
- **Eliminar X**: buscar X con puntero previo para “saltar” el nodo y liberar memoria.
- **Eliminar antes de X**: requiere tres punteros (R, T, Q) para rastrear y reconectar correctamente.
- **Eliminar después de X**: localizar X, enlazar con el “nieto” y liberar el siguiente.

Actividades del laboratorio

- **Actividad 1: Estructura base**
  - Defina la estructura Nodo con campos info y liga.
  - Implemente funciones:
    - insertar_inicio(P, dato)
    - imprimir(P)
- **Actividad 2: Inserciones**
  - Implemente y pruebe:
    - insertar_despues_X(P, X, dato)
    - Pruebas: insertar en lista vacía, insertar cuando X no existe, insertar cuando X es el último.
- **Actividad 3: Eliminaciones básicas**
  - Implemente y pruebe:
    - elimina_inicio(P)
    - elimina_ultimo(P) Pruebas obligatorias: lista vacía, lista con un solo nodo, lista con múltiples nodos.
- **Actividad 4: Eliminaciones avanzadas**
  - Implemente y pruebe:
    - elimina_X(P, X)
    - elimina_antes_X(P, X)
    - elimina_despues_X(P, X) Incluya validaciones y mensajes cuando X no exista o no aplique el caso.

Ejercicios prácticos para verificación (Prueba de Escritorio)

1. Dada la lista inicial: **10 → 20 → 30 → 40 → NIL**
    - Inserte 25 después de 20.
    - Elimine el nodo con valor 10.
    - Elimine el nodo anterior a 30.
    - Elimine el nodo posterior a 30.
    - Muestre la lista tras cada operación.
2. Explique con un diagrama cómo se reconectan los punteros en elimina_antes_X.

Entregables

- Código fuente comentado (C o C++).
- Capturas de salida del programa.
- Respuestas a los ejercicios de análisis.

Criterios de evaluación (rúbrica)	
Correctitud funcional	40%
Manejo de casos de borde	20%
Claridad del código y comentarios	20%
Análisis y explicación	20%

Notas finales

Se recomienda verificar cada operación con la función imprimir y evitar *memory leaks* validando punteros antes de liberar memoria.

una preve explicacion de lo que
hace el codigo:

Este codigo maneja una fila de numeros enlazados uno detras del otro. Podes meter numeros al principio, meter despues de cualquie otro, sacar el primer, sacar el ultimpo, o sacar un numero en especifico. Tambien podes sacar el que esta antes o el que esta despues de un numero que elijes. El programa te muestras un menu para que esocojas que quieres hacer y te va enseñando la fila despues de cada jugada. Es como tener una lista de espera que vos podes modificara a tu antojo.

prueva de consola:

```
--- QUE QUIERES HACER ---
1. Meter al principio
2. Meter despues de X
3. Sacar el primero
4. Sacar el ultimo
5. Sacar un numero X
6. Sacar el que ta ANTES de X
7. Sacar el que ta DESPUES de X
8. Ver la fila
9. Salir
Elige: 1
  Numero a meter: 20
  Metido 20 al principio.
  20 -> 30 -> 40 -> NIL

--- QUE QUIERES HACER ---
1. Meter al principio
2. Meter despues de X
3. Sacar el primero
4. Sacar el ultimo
5. Sacar un numero X
6. Sacar el que ta ANTES de X
7. Sacar el que ta DESPUES de X
8. Ver la fila
9. Salir
Elige: 1
  Numero a meter: 10
  Metido 10 al principio.
  10 -> 20 -> 30 -> 40 -> NIL

--- QUE QUIERES HACER ---
1. Meter al principio
2. Meter despues de X
3. Sacar el primero
4. Sacar el ultimo
5. Sacar un numero X
6. Sacar el que ta ANTES de X
7. Sacar el que ta DESPUES de X
8. Ver la fila
9. Salir
Elige:
```

```
--- QUE QUIERES HACER ---
1. Meter al principio
2. Meter despues de X
3. Sacar el primero
4. Sacar el ultimo
5. Sacar un numero X
6. Sacar el que ta ANTES de X
7. Sacar el que ta DESPUES de X
8. Ver la fila
9. Salir
Elige: 2
  Meter despues de: 20
  Numero a meter: 25
  Metido 25 despues de 20
  10 -> 20 -> 25 -> 30 -> 40 -> NIL
```

```
  10 -> 20 -> 25 -> 30 -> 40 -> NIL

--- QUE QUIERES HACER ---
1. Meter al principio
2. Meter despues de X
3. Sacar el primero
4. Sacar el ultimo
5. Sacar un numero X
6. Sacar el que ta ANTES de X
7. Sacar el que ta DESPUES de X
8. Ver la fila
9. Salir
Elige: 5
  Numero a sacar: 10
  Sacado 10 del principio
  20 -> 25 -> 30 -> 40 -> NIL
```

```
  20 -> 25 -> 30 -> 40 -> NIL

--- QUE QUIERES HACER ---
1. Meter al principio
2. Meter despues de X
3. Sacar el primero
4. Sacar el ultimo
5. Sacar un numero X
6. Sacar el que ta ANTES de X
7. Sacar el que ta DESPUES de X
8. Ver la fila
9. Salir
Elige: 6
  Sacar el que ta ANTES de: 30
  Sacado 25 (el que taba antes de 30)
  20 -> 30 -> 40 -> NIL
```

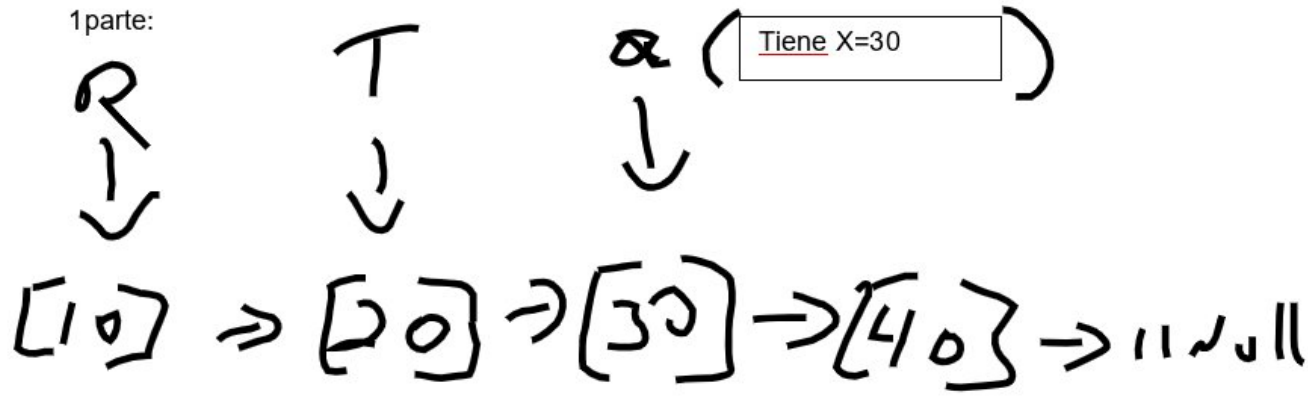
```
  20 -> 30 -> 40 -> NIL

--- QUE QUIERES HACER ---
1. Meter al principio
2. Meter despues de X
3. Sacar el primero
4. Sacar el ultimo
5. Sacar un numero X
6. Sacar el que ta ANTES de X
7. Sacar el que ta DESPUES de X
8. Ver la fila
9. Salir
Elige: 7
  Sacar el que ta DESPUES de: 30
  Sacado 40 (el que taba despues de 30)
  20 -> 30 -> NIL
```

El diagrama de eliminar_antes_X

Esta funcion elimina el nodo que esta antes de un nodo que contieneel valor x

1parte:



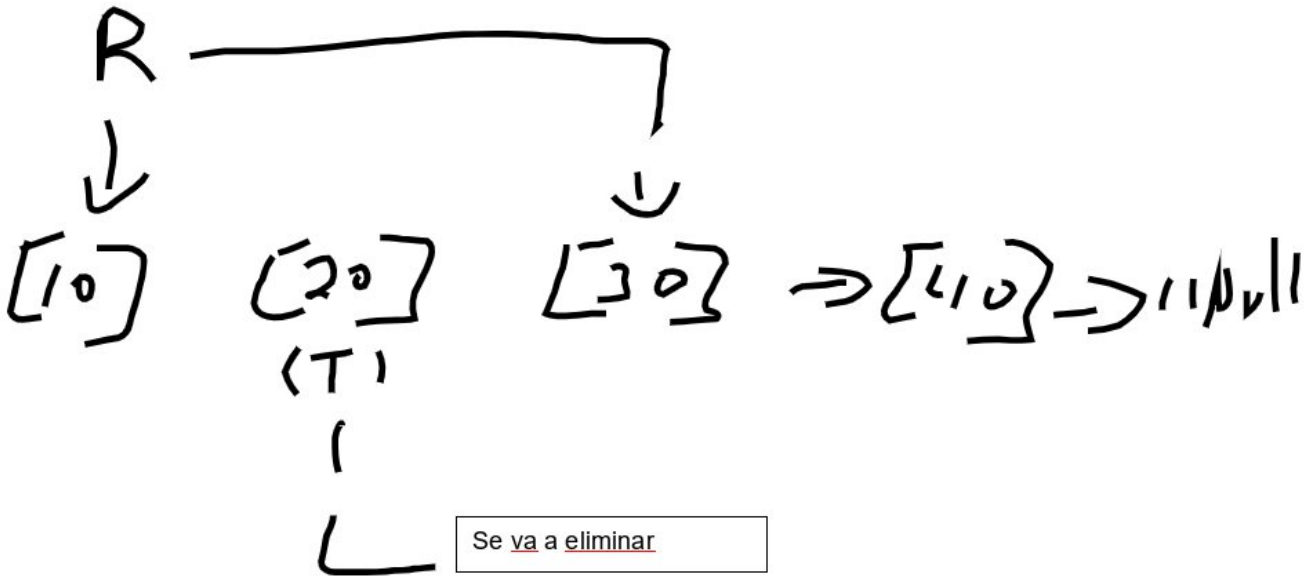
R = apunta al nodo que esta antes del que vamos a eliminiar

T = apunta al nodo que vamos a eliminar(el anterior a X)

Q = apunta al nodo que contine el valor X

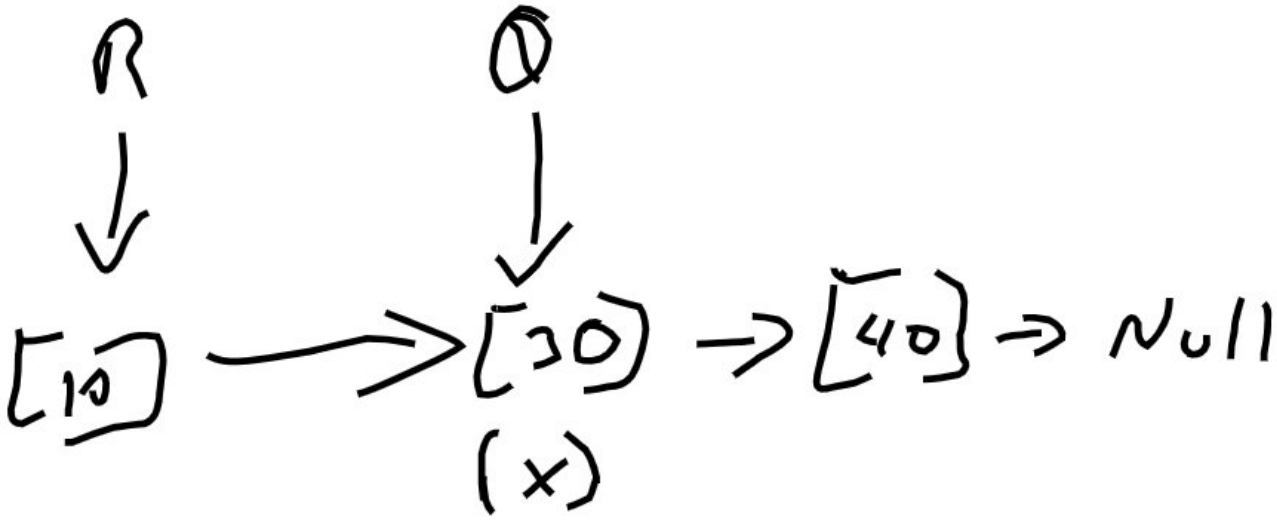
2parte
.

Hacemos que R apunte directamente a Q, saltandonos a T



En el codigo: R -> liga = Q

3parte:



Litas final: [10] -> [30] -> [40]

El nodo 20 ya no exite se libero su memoria